

Proyecto Integrador Ecommify - Investigación

Alexander Caballero Garcia

Astrid Carolina Rodriguez Cristancho

Brian Maldonado

Luis Alberto Rojas Adames

Facultad de Ingeniería, Universidad de La Sabana

Diseño y Optimización de Bases de Datos

Ing. Miguel Alfonso Varela

25 de Mayo de 2026

Resumen

Este documento aborda la diferencia entre cuándo es el momento adecuado para utilizar una base de datos relacional o no relacional, aplicado al caso real de Ecommify. A lo largo del diseño se trabajó la capacidad de identificar cuándo se está frente a una transacción OLAP u OLTP y cómo esa distinción define la arquitectura.

De igual manera, se reforzó el pensamiento analítico en la toma de decisiones relacionadas con rendimiento, calidad, seguridad e integridad de los sistemas, buscando mantener una visión crítica sobre el futuro: cómo los cambios pueden afectar un sistema y si las soluciones propuestas serán capaces de adaptarse a nuevas necesidades.

También se amplían los conocimientos sobre nuevos tipos de datos, nuevas formas de realizar búsquedas y conceptos modernos que permiten consultas más eficientes y precisas. Entre estos temas se incluyen la geolocalización y el uso de extensiones como PostGIS y pg_trgm, aplicadas directamente al dataset Brazilian E-Commerce de Olist sobre el que se construyó toda la arquitectura híbrida PostgreSQL-MongoDB de Ecommify.

Análisis de Requisitos

Contexto de Ecommify

Ecommify es una plataforma de comercio electrónico multivendedor especializada en productos tecnológicos. El sistema permite gestionar clientes, vendedores, órdenes de compra, pagos, catálogo de productos, reseñas y procesos logísticos asociados al comercio electrónico.

Este proyecto toma como base el dataset de Brazilian E-Commerce Public Dataset by Olist, disponible en Kaggle, compuesto por múltiples tablas relacionales que representan transacciones reales de e-commerce realizadas en Brasil entre los años 2016 y 2018.

El objetivo principal para esta unidad consiste en diseñar una arquitectura híbrida de bases de datos utilizando PostgreSQL y MongoDB, capaz de soportar tanto cargas transaccionales (OLTP) como cargas analíticas (OLAP). PostgreSQL se utiliza como motor principal para el módulo transaccional debido a sus garantías ACID, integridad referencial y soporte avanzado para modelado relacional, mientras que MongoDB se proyecta como solución complementaria para módulos semiestructurados y analíticos como catálogo extendido y reseñas.

El diseño propuesto busca garantizar consistencia, escalabilidad y flexibilidad de esquema, aplicando principios de normalización, modelado relacional avanzado y estrategias híbridas de persistencia de datos.

Requisitos Funcionales

El sistema Ecommify debe soportar las operaciones principales de una plataforma de comercio electrónico multivendedor, garantizando integridad transaccional, trazabilidad de pedidos y capacidad analítica sobre el comportamiento comercial. A partir del análisis del dataset Olist y del enfoque arquitectónico seleccionado, se identificaron los siguientes requisitos funcionales relacionados en la Tabla 1:

Tabla 1

Listado requerimientos funcionales

Requerimiento Funcional	Descripción
RF-01. Gestionar clientes y vendedores	El sistema debe permitir registrar, consultar y relacionar clientes y vendedores con información geográfica asociada.
RF-02. Gestionar catálogo de productos	El sistema debe almacenar productos organizados por categorías, incluyendo atributos variables y contenido multimedia asociado.
RF-03. Gestionar órdenes de compra	El sistema debe registrar órdenes realizadas por clientes, incluyendo fechas, estados de procesamiento y trazabilidad logística.
RF-04. Gestionar ítems de órdenes	Cada orden debe permitir múltiples productos asociados a diferentes vendedores, conservando información de precios y costos de envío.
RF-05. Gestionar pagos	El sistema debe registrar métodos de pago, cuotas y valores monetarios asociados a cada orden, garantizando consistencia transaccional.
RF-06. Gestionar reseñas y calificaciones	Los clientes deben poder registrar puntuaciones y comentarios asociados a las órdenes realizadas.
RF-07. Soportar consultas analíticas	El sistema debe permitir análisis sobre ventas, categorías, comportamiento de clientes y desempeño de vendedores.
RF-08. Soportar atributos semiestructurados	El sistema debe manejar atributos variables por categoría de producto utilizando estructuras flexibles como JSONB y documentos MongoDB.
RF-09. Garantizar integridad referencial	Todas las entidades transaccionales deben mantener relaciones consistentes mediante claves primarias y foráneas.

RF-10. Permitir escalabilidad futura	La arquitectura debe facilitar migración parcial de módulos analíticos hacia MongoDB sin afectar el núcleo transaccional
--------------------------------------	--

Requisitos no funcionales

Además de los requisitos funcionales, el diseño de Ecommify debe cumplir restricciones técnicas y operativas relacionadas con rendimiento, consistencia, escalabilidad y mantenibilidad de la arquitectura de datos, por ello los relacionamos en la Tabla 2.

Tabla 2

Listado de requerimientos no funcionales

Requerimiento No Funcional	Descripción
RNF-01. Consistencia transaccional	Las operaciones relacionadas con órdenes, pagos y vendedores deben garantizar propiedades ACID para evitar inconsistencias financieras o pérdida de trazabilidad.
RNF-02. Integridad referencial	El modelo relacional debe garantizar relaciones consistentes mediante claves primarias, claves foráneas y restricciones de integridad.
RNF-03. Escalabilidad	La arquitectura debe permitir el crecimiento progresivo del volumen de datos y facilitar la separación futura entre módulos transaccionales y analíticos.
RNF-04. Flexibilidad de esquema	El sistema debe soportar atributos heterogéneos por categoría de producto mediante tipos avanzados como JSONB y estructuras semiestructuradas.
RNF-05. Rendimiento de consultas	Las consultas frecuentes sobre órdenes, productos y clientes deben optimizarse mediante índices, vistas materializadas y estrategias de particionamiento.
RNF-06. Disponibilidad	Los módulos analíticos y de catálogo deben priorizar disponibilidad y capacidad de consulta aun bajo crecimiento del volumen de datos.
RNF-07. Mantenibilidad	El diseño debe facilitar la evolución del esquema, incorporación de nuevas entidades y mantenimiento de reglas de negocio sin afectar módulos existentes.

RNF-08. Compatibilidad con plataformas gratuitas	La solución debe ajustarse a las limitaciones de Supabase Free Tier, MongoDB Atlas M0 y Google Colab, utilizadas como entorno académico de implementación.
RNF-09. Trazabilidad temporal	Todas las entidades transaccionales deben conservar información temporal utilizando TIMESTAMPTZ para soportar auditoría y análisis histórico.
RNF-10. Seguridad y validación	El modelo debe implementar restricciones CHECK, validaciones y controles básicos de integridad para minimizar registros inválidos.

Entidades Principales

A partir del análisis exploratorio del dataset Brazilian E-Commerce (Olist) y de los requisitos funcionales identificados, se determinaron las principales entidades del dominio necesarias para soportar el modelo transaccional y analítico de Ecommify. La Tabla 3 presenta las entidades principales consideradas en el diseño del sistema.

Tabla 3

Entidades principales del dominio Ecommify

Entidad	Descripción
geolocations	Almacena información geográfica asociada a clientes y vendedores, incluyendo coordenadas y ubicación.
customers	Representa a los clientes registrados que realizan órdenes dentro de la plataforma.
sellers	Representa a los vendedores asociados al marketplace Ecommify.
categories	Clasificación de productos por categoría comercial.
products	Contiene la información básica de los productos comercializados.
product_details	Almacena atributos extendidos y variables de productos utilizando JSONB.
orders	Representa las órdenes de compra realizadas por los clientes.
order_items	Entidad débil que almacena los productos asociados a cada orden junto con vendedor, precio y costo de envío.

order_payments	Registra la información de pagos asociados a las órdenes.
reviews	Contiene calificaciones y comentarios realizados por los clientes sobre sus órdenes.

Estas entidades conforman el núcleo transaccional del sistema implementado sobre PostgreSQL. Adicionalmente, algunas entidades como product_details y reviews se consideran candidatas futuras para migración parcial hacia MongoDB debido a su naturaleza semiestructurada y analítica.

Volumen esperado de datos

El diseño de Ecommify considera tanto el volumen actual del dataset Brazilian E-Commerce (Olist) como escenarios futuros de crecimiento transaccional y analítico. Durante el análisis exploratorio de datos realizado en Google Colab se identificó que el dataset contiene aproximadamente:

Tabla 4

Volumen aproximado de entidades del dataset Olist

Entidad	Volumen aproximado
orders	99.441 registros
customers	99.441 registros
sellers	3.095 registros
products	32.951 registros
order_items	113.425 registros
order_payments	103.887 registros
reviews	99.992 registros

Las relaciones entre tablas fueron validadas mediante operaciones de merge y verificación de claves foráneas en Google Colab, confirmando

consistencia estructural entre entidades transaccionales y ausencia de registros huérfanos en las principales relaciones del dataset.

Análisis Exploratorio de Datos

El análisis exploratorio completo del dataset Brazilian E-Commerce (Olist) fue realizado en Google Colab y se documenta en detalle en las entregas realizadas en la unidad 1, del presente proyecto integrador. El notebook completo está disponible en el repositorio GitHub del proyecto. Los hallazgos principales que fundamentan las decisiones de diseño de este documento se resumen a continuación:

La Tabla 5 presenta los hallazgos de calidad de datos identificados durante el EDA y su impacto en las decisiones de diseño.

Tabla 5

Hallazgos del EDA y su impacto en el diseño

Hallazgo	Dato	Decisión de diseño derivada
Órdenes sin fecha de entrega al cliente	2.965 registros (3%)	delivered_customer_date TIMESTAMPTZ NULL
Órdenes sin fecha de despacho al transportista	1.783 registros (1.8%)	delivered_carrier_date TIMESTAMPTZ NULL
Productos sin categoría asignada	610 registros (1.9%)	category_id UUID NULL en products
Reseñas sin título de comentario	87.656 registros (88%)	comment_title VARCHAR(200) NULL
Reseñas sin mensaje	58.247 registros (59%)	comment_message TEXT NULL
Registros de geolocalización	1,000,163 filas para ~19,000	Deduplicación por

por prefijo postal	zips únicos	zip_code_prefix + UUID sintético como PK
Concentración geográfica de clientes	SP concentra el 42% de clientes	Índice en geolocation_id para optimizar consultas regionales
Crecimiento temporal del volumen	Pico en mayo y agosto 2018	Particionamiento por año en tabla orders

Para el análisis completo con visualizaciones de distribución temporal, distribución geográfica y validación de relaciones entre tablas, ver **Actividad 1, Apéndice A** y el notebook `Data_Exploration_Analysis.ipynb` en el repositorio GitHub.

Diseño Conceptual

El diseño conceptual de Ecommify está construido a partir del análisis funcional del dominio de comercio electrónico y del estudio estructural del dataset Brazilian E-Commerce (Olist). Esta fase representa las entidades principales del negocio, sus relaciones, cardinalidades y restricciones fundamentales, de forma independiente a los detalles de implementación física.

El modelo fue desarrollado bajo un enfoque Entidad-Relación (ER), considerando los siguientes principios de diseño:

- Separación entre entidades fuertes e identidad propia mediante UUID sintético
- Entidades débiles cuya existencia depende de una entidad padre
- Soporte para arquitectura híbrida PostgreSQL - MongoDB
- Escalabilidad futura y compatibilidad con tipos avanzados de PostgreSQL

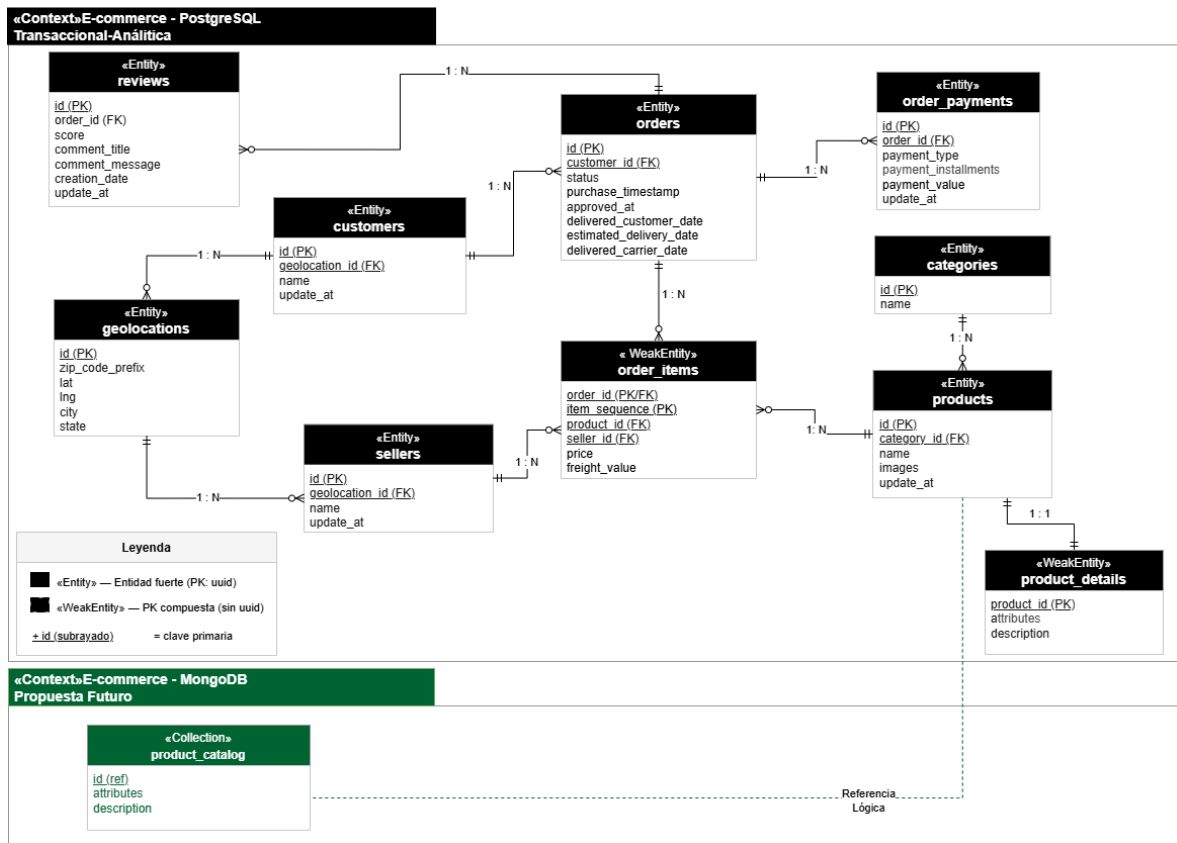
Modelo Entidad Relación

La Figura 1 presenta el modelo ER completo de Ecommify. El diagrama distingue dos contextos de persistencia mediante notación de colores: el módulo transaccional en PostgreSQL y el módulo analítico proyectado en MongoDB como propuesta futura.

- UUID como clave primaria sintética en todas las entidades fuertes
- snake_case en nombres de tablas y columnas, compatible con Npgsql y EF Core
- TIMESTAMPTZ en todos los campos de fecha para soporte correcto de zona horaria (dataset Brasil, UTC-3)
- update_at en entidades transaccionales principales para trazabilidad temporal y soporte de triggers

Figura 1

Modelo Entidad-Relación Ecommify E-commerce



Descripción de Entidades

El modelo identifica diez entidades distribuidas entre entidades fuertes y entidades débiles. La eliminación de `product_images` como entidad separada y su consolidación como campo `images: text[]` dentro de `products` responde a la aplicación del tipo avanzado ARRAY de PostgreSQL, reduciendo JOINS innecesarios para una relación de alta cohesión.

La Tabla 6 presenta la clasificación, clave primaria y descripción funcional de cada entidad del modelo.

Tabla 6

Descripción de entidades del modelo conceptual de Ecommify

Entidad	Tipo	PK	Descripción
geolocations	Fuerte	id uuid	Almacena información geográfica de clientes y vendedores. Contiene coordenadas (lat, lng), ciudad y estado. El dataset original tiene 1,000,163 registros; se aplica deduplicación por zip_code_prefix para el entorno académico resultando en ~19,000 registros único
customers	Fuerte	id uuid	Clientes registrados en la plataforma. Referencia geolocations mediante FK eliminando redundancia de city y state, cumpliendo 3FN
sellers	Fuerte	id uuid	Vendedores del marketplace. Referencia geolocations de forma análoga a customers
categories	Fuerte	id uuid	Clasificación comercial de productos. Tabla pequeña (~71 registros) con restricción UNIQUE sobre name
products	Fuerte	id uuid	Información base de productos. Incluye campo <code>images: text[]</code> como array nativo de PostgreSQL para URLs de imágenes, eliminando la necesidad de una tabla intermedia para esta relación de alta cohesión
product_details	Débil	product_id (FK)	Atributos extendidos y variables de productos mediante JSONB. Relación 1:1 con products, su identidad depende completamente de la entidad padre
orders	Fuerte	id uuid	Órdenes de compra con ciclo de vida completo: <code>purchase_timestamp</code> , <code>approved_at</code> , <code>delivered_carrier_date</code> , <code>delivered_customer_date</code> y <code>estimated_delivery_date</code> . Incluye <code>update_at</code> para trazabilidad
order_items	Débil	(order_id, item_sequence)	Entidad débil con PK compuesta. Almacena productos por orden con vendedor, precio y costo de envío. Su identidad depende de orders
order_payments	Fuerte	id uuid	Métodos de pago, cuotas e importes por orden. Una orden puede tener múltiples registros de pago con distintos métodos. Incluye <code>update_at</code> para auditoría financiera
reviews	Fuerte	id uuid	Calificaciones y comentarios de clientes sobre órdenes. El EDA confirma que el 88% no tiene título de comentario, justificando campos opcionales NULL

Relaciones y cardinalidades

El modelo establece nueve relaciones entre entidades, todas implementadas mediante claves foráneas en PostgreSQL. La Tabla 7 documenta cada relación con su cardinalidad y mecanismo de implementación.

Tabla 7

Relaciones y cardinalidades del modelo conceptual

Relación	Cardinalidad	Implementación	Justificación
geolocations y customers	1 : N	FK geolocation_id en customers	Un área geográfica puede asociarse a múltiples clientes
geolocations y sellers	1 : N	FK geolocation_id en sellers	Un área geográfica puede asociarse a múltiples vendedores
customers y orders	1 : N	FK customer_id en orders NOT NULL	Un cliente puede realizar múltiples órdenes; toda orden requiere cliente
orders y order_items	1 : N	FK order_id en order_items NOT NULL	Toda orden contiene uno o más ítems; los ítems no existen sin orden
products y order_items	1 : N	FK product_id en order_items NOT NULL	Un producto puede aparecer en ítems de múltiples órdenes
sellers y order_items	1 : N	FK seller_id en order_items NOT NULL	Un vendedor puede participar en ítems de múltiples órdenes
categories y products	1 : N	FK category_id en products NULL	Una categoría agrupa múltiples productos; nullable porque 610 productos del dataset no tienen categoría asignada

products y product_details	1 : 1	product_id es PK y FK en product_details	Separación de atributos descriptivos pesados (JSONB) de la tabla transaccional principal
orders y order_payments	1 : N	FK order_id en order_payments NOT NULL	Una orden puede pagarse con múltiples métodos simultáneos
orders y reviews	1 : N	FK order_id en reviews NOT NULL	Una orden puede recibir múltiples calificaciones a lo largo de su ciclo de vida

Restricciones de negocio

Las restricciones de negocio fueron identificadas a partir de un análisis funcional del dominio y del comportamiento observado en el dataset Olist durante el EDA. La Tabla 8 documenta cada restricción con su entidad, descripción e implementación técnica.

Tabla 8

Restricciones de negocio del modelo conceptual de Ecommify

Restricción	Entidad	Descripción	Implementación
RN-01. Toda orden requiere cliente	orders	No puede existir una orden sin cliente registrado en el sistema	FK customer_id NOT NULL + REFERENCES customers
RN-02. Todo ítem requiere orden, producto y vendedor	order_items	Los tres campos son obligatorios para garantizar trazabilidad completa del flujo comercial	FK order_id, product_id, seller_id NOT NULL
RN-03. Los valores monetarios no pueden ser negativos	order_items	Precio y flete deben ser mayores o iguales a cero	CHECK (price >= 0), CHECK (freight_value >= 0)
RN-04. El score de reseña es una escala cerrada 1-5	reviews	El sistema de calificación acepta únicamente valores enteros entre 1 y 5	SMALLINT + CHECK (score BETWEEN 1 AND 5)
RN-05. El estado de	orders	Solo se permiten los ocho	CHECK (status IN

orden debe ser un valor del ciclo de vida		estados definidos del flujo operativo	('created', 'approved', 'invoiced', 'processing', 'shipped', 'delivered', 'unavailable', 'canceled'))
RN-06. El nombre de categoría debe ser único	categories	No pueden coexistir dos categorías con el mismo nombre comercial	UNIQUE (name)
RN-07. Las coordenadas geográficas son obligatorias	geolocations	lat y lng son campos NOT NULL para habilitar soporte geoespacial futuro con PostGIS	lat NOT NULL, lng NOT NULL
RN-08. El pago no puede ser negativo	order_payments	El valor del pago y las cuotas deben ser no negativos	CHECK (payment_value >= 0), CHECK (payment_installments >= 0)
RN-09. Los timestamps usan zona horaria explícita	Todas	El dataset proviene de Brasil (UTC-3); usar TIMESTAMP sin zona generaría errores silenciosos de conversión	TIMESTAMPTZ en todos los campos de fecha y hora
RN-10. Trazabilidad temporal en entidades transaccionales	orders, order_payments	Las entidades con impacto financiero deben registrar la última modificación para auditoría y sincronización	update_at TIMESTAMPTZ NOT NULL en orders y order_payments

Diseño Lógico PostgreSQL

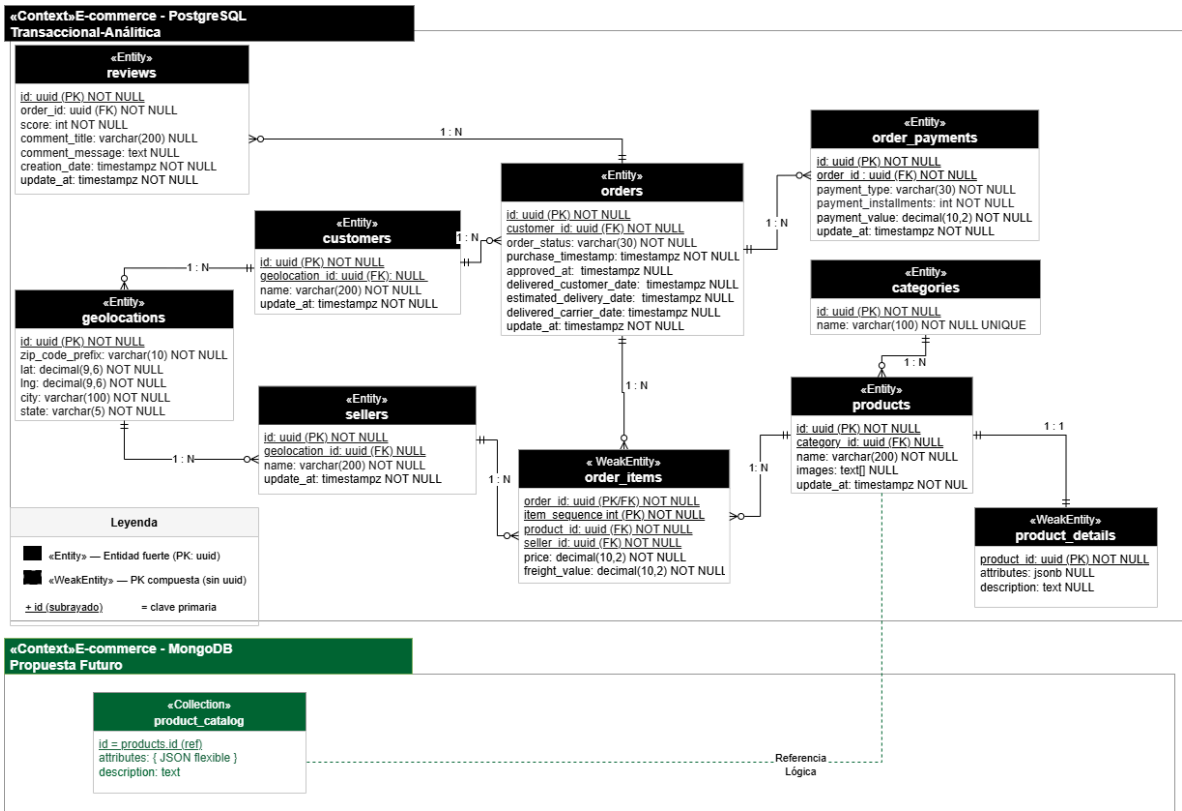
El diseño lógico del módulo transaccional de Ecommify traduce el modelo conceptual a un esquema relacional implementable en PostgreSQL, aplicando normalización hasta 3FN, tipos de datos nativos avanzados y restricciones de integridad explícitas.

Esquema relacional normalizado

El diseño lógico del módulo transaccional de Ecommify traduce el modelo conceptual a un esquema relacional implementable en PostgreSQL, aplicando normalización hasta 3FN, tipos de datos nativos avanzados y restricciones de integridad explícitas.

Figura 2

Esquema Lógico Relacional Módulo Transaccional PostgreSQL



Diccionario de Tablas

Las siguientes tablas documentan la estructura física de cada entidad del módulo transaccional. Para cada tabla se especifican columnas, tipos de datos, restricciones y justificación de las decisiones de diseño relevantes.

Claves primarias y foráneas.

La Tabla 9 describe la estructura de la tabla geolocations, responsable del almacenamiento de información geográfica de clientes y vendedores.

Tabla 9

Diccionario de datos geolocations

Columna	Tipo	Nullable	Constraint	Descripción
id	UUID	NOT NULL	PK, DEFAULT gen_random_u uid()	Identificador sintético único. UUID garantiza inmutabilidad ante cambios en códigos postales
zip_code_pre fix	VARCHAR(10)	NOT NULL	INDEX	Prefijo para el código postal. Atributo de búsqueda, no clave y permite múltiples coordenadas por prefijo
lat	DECIMAL(9,6)	NOT NULL		Latitud geográfica. NOT NULL requerido para soporte geoespacial futuro con PostGIS
lng	DECIMAL(9,6)	NOT NULL		Longitud geográfica. NOT NULL requerido para soporte geoespacial futuro con PostGIS
city	VARCHAR(100)	NOT NULL		Nombre de la ciudad asociada al prefijo postal
state	VARCHAR(5)	NOT NULL		Código del estado o provincia. VARCHAR(5) permite escalabilidad a otros países con códigos más largos

La Tabla 10 describe la estructura de la tabla customers, que representa a los clientes registrados en la plataforma.

Tabla 10

Diccionario de datos customers

Columna	Tipo	Nullable	Constraint	Descripción
id	UUID	NOT NULL	PK, DEFAULT gen_random_uuid()	Identificador sintético del cliente
geolocation_id	UUID	NULL	FK, geolocations(id)	Referencia a la ubicación geográfica. NULL permite registrar clientes sin geolocalización validada
name	VARCHAR(200)	NOT NULL		Nombre del cliente

La Tabla 11 describe la estructura de la tabla sellers, que representa a los vendedores del marketplace.

Tabla 11

Diccionario de datos sellers

Columna	Tipo	Nullable	Constraint	Descripción
id	UUID	NOT NULL	PK, DEFAULT gen_random_uuid()	Identificador sintético del vendedor
geolocation_id	UUID	NULL	FK, geolocations(id)	Referencia geográfica del vendedor. Mismo patrón que customers
name	VARCHAR(200)	NOT NULL		Nombre o razón social del vendedor

La Tabla 12 describe la estructura de la tabla categories, que clasifica los productos del catálogo.

Tabla 12

Diccionario de datos categories

Columna	Tipo	Nullable	Constraint	Descripción
id	UUID	NOT NULL	PK, DEFAULT gen_random_uuid()	Identificador sintético de la categoría
name	VARCHAR(100)	NOT NULL	UNIQUE	Nombre único de la categoría. UNIQUE previene duplicados que generarían productos mal clasificados

La Tabla 13 describe la estructura de la tabla products, entidad central del catálogo de Ecommify.

Tabla 13*Diccionario de datos products*

Columna	Tipo	Nullable	Constraint	Descripción
id	UUID	NOT NULL	PK, DEFAULT gen_random_uuid()	Identificador sintético del producto
category_id	UUID	NULL	FK, categories(id)	Clasificación del producto. NULL porque el EDA confirmó 610 productos sin categoría en el dataset
name	VARCHAR(200)	NOT NULL		Nombre comercial del producto
images	TEXT []	NULL		Array de URLs de imágenes del producto. Tipo avanzado ARRAY elimina la necesidad de tabla intermedia para esta relación de alta cohesión

La Tabla 14 describe la estructura de product_details, entidad débil que almacena atributos extendidos de productos.

Tabla 14*Diccionario de datos product_details*

Columna	Tipo	Nullable	Constraint	Descripción
product_id	UUID	NOT NULL	PK, FK, products(id)	PK y FK simultáneamente. La identidad de esta entidad depende completamente de products
attributes	JSONB	NULL	GIN INDEX	Atributos variables por categoría (laptop: RAM/CPU; audífonos: impedancia/frecuencia). JSONB permite consultas e índices sobre campos internos del documento
description	TEXT	NULL		Descripción textual extendida del producto

La Tabla 15 describe la estructura de la tabla orders, núcleo transaccional del sistema.

Tabla 15*Diccionario de datos orders*

Columna	Tipo	Nullable	Constraint	Descripción
id	UUID	NOT NULL	PK, DEFAULT gen_random_ uuid()	Identificador sintético de la orden
customer_id	UUID	NOT NULL	FK, customers(id)	Toda orden requiere cliente. NOT NULL garantiza integridad referencial
order_status	VARCHAR(30)	NOT NULL	CHECK	Estado del ciclo de vida. CHECK restringe a los 8 valores válidos del flujo operativo
purchase_timestamp	TIMESTAMPTZ	NOT NULL	INDEX	Fecha y hora de creación de la orden. Índice para consultas temporales y particionamiento
approved_at	TIMESTAMPTZ	NULL		Fecha de aprobación. NULL porque 160 órdenes del dataset no tienen aprobación registrada
delivered_carrier_date	TIMESTAMPTZ	NULL		Fecha de entrega al transportista. NULL porque 1,783 órdenes no tienen este campo
delivered_customer_date	TIMESTAMPTZ	NULL		Fecha de entrega al cliente. NULL porque 2,965 órdenes no tienen entrega confirmada
estimated_delivery_date	TIMESTAMPTZ	NULL		Fecha estimada de entrega según el vendedor
update_at	TIMESTAMPTZ	NOT NULL	DEFAULT NOW()	Timestamp de última modificación. Mantenido por trigger para auditoría del ciclo de vida

La Tabla 16 describe la estructura de order_items, entidad débil que relaciona órdenes con productos y vendedores.

Tabla 16

Diccionario de datos order_items

Columna	Tipo	Nullable	Constraint	Descripción
order_id	UUID	NOT NULL	PK (parte 1), FK, orders(id)	Primera parte de la PK compuesta. Identifica la orden padre
item_sequence	INT	NOT NULL	PK (parte 2)	Segunda parte de la PK compuesta. Secuencia del ítem dentro de la orden
product_id	UUID	NOT NULL	FK, products(id)	Producto asociado al ítem. NOT NULL para garantizar trazabilidad
seller_id	UUID	NOT NULL	FK, sellers(id)	Vendedor responsable del ítem. NOT NULL para facturación y comisiones
price	DECIMAL(10,2)	NOT NULL	CHECK (>= 0)	Precio unitario del producto en la orden
freight_value	DECIMAL(10,2)	NOT NULL	CHECK (>= 0)	Costo de envío asociado al ítem

La Tabla 17 describe la estructura de order_payments, que registra los pagos asociados a cada orden.

Tabla 17

Diccionario de datos order_payments

Columna	Tipo	Nullable	Constraint	Descripción
id	UUID	NOT NULL	PK, DEFAULT gen_random_uuid()	Identificador sintético del pago
order_id	UUID	NOT NULL	FK orders(id), INDEX	Orden asociada al pago. NOT NULL porque un pago sin orden es inválido
payment_type	VARCHAR(30)	NOT NULL		Método de pago (credit_card, boleto, voucher, debit_card)
payment_installments	INT	NOT NULL	CHECK (>= 0)	Número de cuotas del pago. Dataset muestra hasta 24 cuotas

payment_value	DECIMAL(10,2)	NOT NULL	CHECK (>= 0)	Valor monetario del pago. No puede ser negativo
update_at	TIMESTAMPTZ	NOT NULL	DEFAULT NOW()	Timestamp de última modificación para auditoría financiera

La Tabla 18 describe la estructura de la tabla reviews, que almacena calificaciones y comentarios de clientes.

Tabla 18

Diccionario de datos reviews

Columna	Tipo	Nullable	Constraint	Descripción
id	UUID	NOT NULL	PK, DEFAULT gen_random_uuid()	Identificador sintético de la reseña
order_id	UUID	NOT NULL	FK orders(id), INDEX	Orden evaluada. NOT NULL porque toda reseña debe referenciar una orden
score	INT	NOT NULL	CHECK (BETWEEN 1 AND 5)	Calificación del cliente en escala 1-5
comment_title	VARCHAR(200)	NULL		Título opcional del comentario. El EDA confirma que el 88% de registros no tiene título
comment_message	TEXT	NULL		Texto libre del comentario. El 59% de registros no tiene mensaje
creation_date	TIMESTAMPTZ	NOT NULL		Fecha de creación de la reseña

Tipos avanzados PostgreSQL

Una de las ventajas distintivas de PostgreSQL frente a sistemas relacionales tradicionales es su soporte nativo para tipos de datos avanzados que permiten modelar estructuras semiestructuradas, colecciones y rangos temporales directamente en el motor relacional. La Tabla 19 documenta los tipos avanzados aplicados en el modelo de Ecommify con su justificación técnica.

La Tabla 19 presenta los tipos avanzados de PostgreSQL aplicados en el modelo de Ecommify, con el campo, la justificación y las consultas representativas habilitadas por cada tipo.

Tabla 19

Tipos avanzados de PostgreSQL aplicados en Ecommify

Tipo	Tabla.Columna	Justificación	Consulta representativa habilitada
JSONB	product_details.attributes	Los atributos de producto varían radicalmente por categoría (laptop: RAM/CPU/pantalla; audífonos: impedancia/frecuencia). JSONB permite almacenar atributos heterogéneos sin columnas nulas ni el anti-patrón EAV. Soporta índices GIN sobre campos internos	SELECT * FROM product_details WHERE attributes @> '{"ram_gb": 16}'
TEXT []	products.images	Un producto tiene múltiples imágenes con orden de visualización. TEXT [] elimina la necesidad de tabla intermedia para esta relación de alta cohesión. El acceso a imágenes siempre ocurre junto con el producto	SELECT name, images[1] AS portada FROM products WHERE id = \$1

TIMESTAMPTZ	Todos los campos de fecha	El dataset Olist proviene de Brasil (UTC-3). TIMESTAMPTZ almacena con zona horaria explícita, evitando errores silenciosos de conversión en consultas cross-timezone	SELECT * FROM orders WHERE purchase_timestamp AT TIME ZONE 'America/Bogota' > NOW() - INTERVAL '30 days'
DECIMAL(10,2)	price, freight_value, payment_value	Valores monetarios requieren precisión exacta. DECIMAL evita errores de redondeo de punto flotante (FLOAT/DOUBLE) que son inaceptables en datos financieros	SELECT SUM(payment_value) FROM order_payments WHERE order_id = \$1
SMALLINT	reviews.score	El score acepta únicamente valores 1-5. SMALLINT (2 bytes) es más eficiente que INT (4 bytes) para este dominio acotado y reduce el footprint en tablas con alto volumen de reseñas	SELECT AVG(score) FROM reviews WHERE order_id = \$1

Consideración de tipos futuros

Para una fase de escalabilidad, se identificaron dos tipos adicionales de PostgreSQL aplicables a Ecommify:

```
-- TSTZRANGE: modelar períodos promocionales de productos

ALTER TABLE product_details ADD COLUMN promotion_period TSTZRANGE NULL;

-- Consulta: productos en promoción activa

SELECT p.name FROM products p JOIN product_details pd ON p.id = pd.product_id WHERE
pd.promotion_period @> NOW();

-- PostGIS GEOMETRY: distancia real vendedor-cliente

ALTER TABLE geolocations ADD COLUMN location GEOMETRY(POINT, 4326);

-- Consulta: distancia en km entre vendedor y cliente
```



```
SELECT ST_Distance(sg.location::geography, cg.location::geography) / 1000 AS
distancia_km FROM sellers s JOIN geolocations sg ON s.geolocation_id = sg.id JOIN
customers c ON c.id = $1 JOIN geolocations cg ON c.geolocation_id = cg.id;
```

Extensiones PostgreSQL

PostgreSQL permite ampliar sus capacidades nativas mediante extensiones que añaden tipos de datos, funciones y operadores especializados sin modificar el motor base. Para Ecommify se evaluaron cuatro extensiones clave: PostGIS, pg_trgm, hstore y pgcrypto. La Tabla 20 presenta una visión general antes de profundizar en cada una.

Tabla 20

Extensiones PostgreSQL evaluadas para Ecommify

Extensión	Tipo	Función Principal	Caso de uso en Ecommify
PostGIS	Extensión Geoespacial	Geometrías, distancias y operaciones espaciales sobre coordenadas	Calcular distancias reales vendedor–cliente para optimizar costos de envío
pg_trgm	Búsqueda de texto	Similitud de cadenas mediante trigramas; búsqueda tolerante a errores tipográficos	Buscar productos aunque el usuario escriba 'notboook' en lugar de 'notebook'
hstore	Clave-Valor	Almacenamiento de pares clave-valor en una sola columna	Atributos simples y planos de productos donde JSONB sería innecesariamente costoso
pgcrypto	Seguridad y Cifrado	Cifrado simétrico, hashing y generación de UUIDs criptográficamente seguros	Proteger datos sensibles de clientes y tokens de pago; generar UUID v4 con gen_random_uuid()

PostGIS: Extensión Geoespacial

PostGIS añade soporte completo para objetos geoespaciales a PostgreSQL: tipos de datos como GEOMETRY y GEOGRAPHY, índices espaciales GiST/BRIN y más de 1,000 funciones para cálculo de distancias, intersecciones, áreas y rutas. La extensión cumple con el estándar OGC (Open Geospatial Consortium) y es considerada la implementación de datos espaciales más completa en el ecosistema relacional open-source (PostgreSQL Global Development Group, 2025).

El dataset Olist contiene coordenadas de latitud y longitud para clientes y vendedores en la tabla geolocations (campos lat y lng). En el diseño actual estos campos son DECIMAL(9,6), lo que limita el cálculo de distancias a estimaciones euclidianas imprecisas. Con PostGIS se puede añadir una columna GEOMETRY(POINT, 4326) y calcular distancias geodésicas reales entre vendedores y clientes para optimizar el costo de envío.

Implementación propuesta en Ecommify:

```
-- Activar extensión

CREATE EXTENSION IF NOT EXISTS postgis;

-- Añadir columna espacial a geolocations

ALTER TABLE geolocations

    ADD COLUMN location GEOMETRY(POINT, 4326);

-- Poblar con datos existentes de lat/lng

UPDATE geolocations

SET location = ST_SetSRID(ST_MakePoint(lng, lat), 4326);

-- Índice espacial GiST para búsquedas por proximidad
```

```

CREATE INDEX idx_geolocations_location

ON geolocations USING GIST(location);

-- Consulta: distancia en km entre vendedor y cliente

SELECT

    s.name AS vendedor,

    c.name AS cliente,

    ROUND(ST_Distance(sg.location::geography, cg.location::geography) / 1000, 2 )

    AS distancia_km

FROM order_items oi

JOIN sellers s ON oi.seller_id = s.id

JOIN geolocations sg ON s.geolocation_id = sg.id

JOIN orders o ON oi.order_id = o.id

JOIN customers c ON o.customer_id = c.id

JOIN geolocations cg ON c.geolocation_id = cg.id

WHERE oi.order_id = $1;

```

El campo lat/lng NOT NULL ya está definido en la restricción RN-07 del modelo conceptual precisamente para habilitar soporte geoespacial futuro. PostGIS es la evolución natural de esa decisión. La distancia real (geodésica) entre vendedor y cliente es insumo para algoritmos de optimización logística, algo que no es posible con álgebra relacional estándar.

pg_trgm: Búsqueda de Texto Tolerante a Errores

pg_trgm implementa búsqueda de similitud de cadenas basada en trigramas (substrings de 3 caracteres consecutivos). La función similarity(a, b)

retorna un valor entre 0 y 1 que indica qué tan similares son dos cadenas. Un umbral de 0.3 es suficiente para capturar errores tipográficos comunes. La extensión incluye el operador % para uso en índices GIN y GiST (PostgreSQL Global Development Group, 2025).

Los usuarios de un marketplace teclean nombres de productos con errores frecuentes: "notboook" en lugar de "notebook", "iphon" en lugar de "iphone", "audifonos" en lugar de "audífonos". Sin pg_trgm, una búsqueda por LIKE o ILIKE no encontraría ningún resultado. Con trigramas se pueden rankear los productos más similares al término buscado.

```
-- Activar extensión

CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Índice GIN para búsqueda tolerante a errores en nombre de producto

CREATE INDEX idx_products_name_trgm ON products USING GIN(name gin_trgm_ops);

-- Consulta: búsqueda tolerante a error tipográfico

SELECT

    id,

    name,

    similarity(name, $1) AS score

FROM products

WHERE similarity(name, $1) > 0.3

ORDER BY score DESC LIMIT 10;

-- Ejemplo: $1 = 'notboook pro 15' -- Retorna: 'Notebook Pro 15' con score ~0.65
```

La búsqueda tolerante a errores tipográficos es un requisito de usabilidad en cualquier catálogo de e-commerce. La alternativa (Elasticsearch o Atlas Search) añade complejidad de infraestructura. pg_trgm resuelve el 80% de los casos de uso de búsqueda tolerante a errores directamente en PostgreSQL sin servicios adicionales, manteniendo la arquitectura simple en el entorno académico con Supabase.

hstore: Almacenamiento Clave-Valor

hstore es una extensión de PostgreSQL que permite almacenar pares clave-valor arbitrarios en una sola columna de tipo hstore. Soporta índices GiST y GIN sobre sus claves y valores, y operadores propios para consulta, inserción y eliminación de pares. Es históricamente anterior a JSONB y más limitado en expresividad: no soporta anidamiento ni arrays (PostgreSQL Global Development Group, 2025).

Para Ecommify, hstore es apropiado para atributos simples y planos que no requieren anidamiento: metadatos de envío, etiquetas de categoría, configuraciones de vendedor. Por ejemplo, un vendedor podría tener atributos como `plan => premium, region => SP, verified => true`.

```
-- Activar extensión

CREATE EXTENSION IF NOT EXISTS hstore;

-- Columna hstore para metadatos planos de vendedor

ALTER TABLE sellers

    ADD COLUMN metadata hstore;

-- Insertar metadatos
```

```
UPDATE sellers

SET metadata = 'plan=>premium, region=>SP, verified=>true'::hstore

WHERE id = $1;

-- Consulta: vendedores premium de São Paulo

SELECT name

FROM sellers

WHERE metadata -> 'plan' = 'premium'

AND metadata -> 'region' = 'SP';
```

JSONB es superior en expresividad (anidamiento, arrays, tipos numéricos reales). hstore tiene menor overhead de almacenamiento para atributos planos simples. Para Ecommify, product_details.attributes usa JSONB porque los atributos de productos son jerárquicos (frecuencia min/max, dimensiones). hstore se reserva para metadatos simples de vendedores o configuraciones de sistema donde el anidamiento no es necesario.

pgcrypto. Seguridad y Cifrado

pgcrypto provee funciones criptográficas para PostgreSQL: hash MD5/SHA, cifrado simétrico AES, cifrado asimétrico con PGP y generación de valores aleatorios seguros. La función gen_random_uuid() genera UUID v4 criptográficamente seguros (PostgreSQL Global Development Group, 2025). En PostgreSQL 13+ gen_random_uuid() es parte de las funciones built-in, pero pgcrypto añade las capacidades de cifrado adicionales.

En el contexto académico con Supabase, pgcrypto tiene dos aplicaciones concretas: garantizar que los UUID de todas las tablas son criptográficamente aleatorios e impredecibles, evitando enumeración de recursos; y en una fase de producción, cifrar datos sensibles como tokens de pago en order_payments.

```
-- Activar extensión

CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- UUID criptográficamente seguro (ya en uso en el diseño)

-- DEFAULT gen_random_uuid() en todas las PKs -- Hash de contraseña con salt (tabla
users futura)

INSERT INTO users (email, password_hash)

VALUES ($1, crypt($2, gen_salt('bf', 10)));

-- Verificar contraseña

SELECT id FROM users WHERE email = $1 AND password_hash = crypt($2, password_hash);

-- Cifrado de dato sensible (token de pago)

UPDATE order_payments

SET payment_token_encrypted = pgp_sym_encrypt( payment_token_plain, $SECRET_KEY )

WHERE id = $1;
```

Supabase activa pgcrypto por defecto en todos sus proyectos, por lo que el costo de adopción es cero. El uso de gen_random_uuid() ya está integrado en el diseño de Ecommify como DEFAULT en todas las PKs. En contexto académico, el valor principal de pgcrypto es documentar la práctica de seguridad que se aplicaría en producción.

Estrategia de Particionamiento

Criterio de particionamiento seleccionado: RANGE por fecha

La tabla `orders` se particiona por `purchase_timestamp` usando el esquema hot/cold que distingue datos activos de datos históricos:

```
-- Tabla padre particionada

CREATE TABLE orders ( id UUID NOT NULL, customer_id UUID NOT NULL REFERENCES
customers(id), order_status VARCHAR(30) NOT NULL CHECK (order_status IN (
'created','approved','invoiced','processing','shipped','delivered','unavailable','can
celed')), purchase_timestamp TIMESTAMPTZ NOT NULL, approved_at TIMESTAMPTZ,
delivered_carrier_date TIMESTAMPTZ, delivered_customer_date TIMESTAMPTZ,
estimated_delivery_date TIMESTAMPTZ, update_at TIMESTAMPTZ NOT NULL DEFAULT NOW() )
PARTITION BY RANGE (purchase_timestamp);

-- Partición HOT: datos del año actual (acceso frecuente)

CREATE TABLE orders_2026 PARTITION OF orders FOR VALUES FROM ('2026-01-01') TO
('2027-01-01');

-- Partición WARM: año anterior

CREATE TABLE orders_2025 PARTITION OF orders FOR VALUES FROM ('2025-01-01') TO
('2026-01-01');

-- Partición COLD: datos históricos del dataset Olist (2016-2018)

CREATE TABLE orders_historico PARTITION OF orders FOR VALUES FROM ('2016-01-01') TO
('2025-01-01');

-- Índices por partición (PostgreSQL los hereda automáticamente)

CREATE INDEX idx_orders_2026_customer ON orders_2026(customer_id);

CREATE INDEX idx_orders_2026_status ON orders_2026(order_status);

CREATE INDEX idx_orders_2026_timestamp ON orders_2026(purchase_timestamp);
```

La Tabla 21 documenta la estrategia de particionamiento y el criterio de clasificación hot/cold.

Tabla 21

Estrategia de particionamiento hot/cold para la tabla orders

Partición	Rango	Tipo	Criterio	Política de mantenimiento
orders_2026	2026-01-01 / 2027-01-01	HOT	Datos activos del año en curso, acceso de lectura y escritura frecuente	VACUUM automático diario
orders_2025	2025-01-01 / 2026-01-01	WARM	Datos del año anterior, consultas analíticas ocasionales	VACUUM semanal, solo lectura
orders_historico	2016-01-01 / 2025-01-01	COLD	Dataset Olist histórico, consultas de auditoría y análisis de tendencias	Archivado mensual, compresión opcional

Beneficios del particionamiento para Ecommify:

- **Partition pruning:** una consulta `WHERE purchase_timestamp > '2026-01-01'` solo escanea `orders_2026`, ignorando el 95% de los datos históricos
- **Mantenimiento granular:** el `VACUUM` y `ANALYZE` pueden ejecutarse por partición sin bloquear toda la tabla
- **Archivado progresivo:** cuando `orders_2025` envejezca, puede moverse a almacenamiento más barato sin afectar la tabla activa.

Índices Iniciales

Los índices fueron diseñados a partir de los patrones de acceso identificados en el análisis de requisitos, priorizando las consultas más frecuentes del flujo transaccional y analítico de Ecommify.

```
-- orders: consultas por cliente, estado y fecha

CREATE INDEX idx_orders_customer_id ON orders(customer_id);
```

```

CREATE INDEX idx_orders_status ON orders(order_status);

CREATE INDEX idx_orders_purchase_date ON orders(purchase_timestamp);

-- order_items: join frecuente con productos y vendedores

CREATE INDEX idx_order_items_product_id ON order_items(product_id);

CREATE INDEX idx_order_items_seller_id ON order_items(seller_id);

-- products: búsqueda por categoría

CREATE INDEX idx_products_category_id ON products(category_id);

-- product_details: búsqueda dentro del JSONB

CREATE INDEX idx_product_details_attributes ON product_details USING GIN(attributes);

-- customers y sellers: join por geolocalización

CREATE INDEX idx_customers_geolocation ON customers(geolocation_id);

CREATE INDEX idx_sellers_geolocation ON sellers(geolocation_id);

-- reviews: consultas por orden

CREATE INDEX idx_reviews_order_id ON reviews(order_id);

-- order_payments: join con órdenes

CREATE INDEX idx_order_payments_order_id ON order_payments(order_id);

-- geolocations: búsqueda por prefijo postal

CREATE INDEX idx_geolocations_zip ON geolocations(zip_code_prefix);

```

La Tabla 22 justifica cada índice según el patrón de consulta que optimiza.

Tabla 22

Justificación de índices iniciales

Índice	Tabla	Patrón de consulta optimizado
--------	-------	-------------------------------

idx_orders_customer_id	orders	Historial de órdenes por cliente, consulta frecuente en perfil de usuario
idx_orders_status	orders	Filtrado por estado del ciclo de vida, dashboard operacional
idx_orders_purchase_date	orders	Consultas temporales y particionamiento, reportes de ventas por período
idx_order_items_product_id	order_items	JOIN orders → order_items → products, detalle de orden
idx_order_items_seller_id	order_items	Ventas por vendedor, reporte de comisiones
idx_products_category_id	products	Catálogo filtrado por categoría, navegación del marketplace
idx_product_details_attributes	product_details	Búsqueda por atributo específico dentro del JSONB (GIN)
idx_customers_geolocation	customers	JOIN geográfico clientes - geolocations
idx_sellers_geolocation	sellers	JOIN geográfico vendedores - geolocations
idx_reviews_order_id	reviews	Reseñas de una orden específica
idx_order_payments_order_id	order_payments	Pagos de una orden específica
idx_geolocations_zip	geolocations	Búsqueda por prefijo postal para consultas de región

Diseño Lógico Preliminar MongoDB

El módulo MongoDB de Ecommify representa la capa analítica y semiestructurada de la arquitectura híbrida. A diferencia del módulo transaccional en PostgreSQL, MongoDB no gestiona flujos de negocio críticos sino entidades cuya naturaleza flexible, volumen analítico o heterogeneidad de atributos hacen inviable o ineficiente el modelo relacional estricto.

Este diseño es preliminar porque en el alcance académico actual todo el dataset Olist se carga en PostgreSQL. La migración hacia MongoDB se plantea

como fase futura cuando el volumen del catálogo o el análisis de reseñas justifique la separación de motores.

Colecciones principales

Se identificaron dos colecciones principales para el módulo MongoDB de Ecommify, basadas en el análisis de requisitos y la matriz de decisión arquitectónica.

La Tabla 23 presenta las colecciones identificadas para MongoDB con su justificación técnica.

Tabla 23

Colecciones principales del módulo MongoDB

Colección	Origen en PostgreSQL	Justificación para MongoDB
product_catalog	products + product_details	Atributos radicalmente heterogéneos por categoría. En PostgreSQL requiere JSONB como workaround. MongoDB los modela nativamente con documentos anidados sin columnas nulas
reviews	reviews	Texto libre, scores y metadatos variables. Alto volumen analítico. No participa en transacciones financieras consistencia eventual aceptable. El Aggregation Pipeline supera a SQL para análisis de sentimiento y distribución de scores

Esquema de Documentos

Colección: product_catalog

El documento de producto en MongoDB consolida la información base del producto con sus atributos variables por categoría, eliminando el JOIN entre `products` y `product_details` que existe en PostgreSQL.

```
// Ejemplo 1 - Producto categoría: electrónica (laptop)

{

  "_id": "550e8400-e29b-41d4-a716-446655440000",

  "product_id": "550e8400-e29b-41d4-a716-446655440000",

  "name": "Notebook Pro 15",

  "category": "informatica_acessorios",

  "images": [ "https://cdn.ecommify.com/products/abc123/img1.jpg",
  "https://cdn.ecommify.com/products/abc123/img2.jpg"

],

  "attributes": {

    "ram_gb": 16,

    "storage_gb": 512,

    "processor": "Intel Core i7-1165G7",

    "screen_inches": 15.6,

    "weight_kg": 1.8,

    "battery_hours": 8

  },

  "description": "Notebook de alto rendimiento para uso profesional",

  "created_at": "2026-01-15T10:30:00Z",

  "updated_at": "2026-03-20T14:22:00Z"

}
```

```
// Ejemplo 2 – Producto categoría: audio (audífonos)
```

```
{  
  
  "_id": "660e8400-e29b-41d4-a716-446655440001",  
  
  "product_id": "660e8400-e29b-41d4-a716-446655440001",  
  
  "name": "Auriculares Pro Bass",  
  
  "category": "audio",  
  
  "images": [ "https://cdn.ecommify.com/products/def456/img1.jpg"  
  
],  
  
  "attributes": {  
  
    "impedance_ohms": 32,  
  
    "frequency_hz": { "min": 20, "max": 20000 },  
  
    "connectivity": "bluetooth",  
  
    "battery_hours": 30,  
  
    "noise_cancelling": true },  
  
    "description": "Auriculares inalámbricos con cancelación de ruido activa",  
  "created_at": "2026-02-10T09:15:00Z",  
  
    "updated_at": "2026-04-01T11:45:00Z"  
  
}
```

```
// Ejemplo 3 – Producto categoría: muebles (silla)
```

```
{  
  
  "_id": "770e8400-e29b-41d4-a716-446655440002",  
  
  "product_id": "770e8400-e29b-41d4-a716-446655440002",  
  
  "name": "Silla Ergonómica Executive",
```

```

    "category": "muebles_escritorio",

    "images": [ "https://cdn.ecommify.com/products/ghi789/img1.jpg",
"https://cdn.ecommify.com/products/ghi789/img2.jpg",
"https://cdn.ecommify.com/products/ghi789/img3.jpg"

    ],

    "attributes": { "max_weight_kg": 120,

    "height_cm": { "min": 45, "max": 55 },

    "material": "metal", "armrests": true,

    "lumbar_support": true, "color": ["negro", "gris", "azul"] },

    "description": "Silla de oficina con soporte lumbar ajustable",

    "created_at": "2026-01-20T08:00:00Z",

    "updated_at": "2026-02-15T16:30:00Z"

}

```

Colección: reviews

El documento de reseña en MongoDB consolida la calificación, el comentario y metadatos analíticos en un solo documento, optimizado para el Aggregation Pipeline.

```

// Ejemplo 1 - Reseña con comentario completo

{

  "_id": "880e8400-e29b-41d4-a716-446655440003",

  "review_id": "880e8400-e29b-41d4-a716-446655440003",

  "order_id": "550e8400-e29b-41d4-a716-000000000001",

  "product_id": "550e8400-e29b-41d4-a716-446655440000",

  "score": 5,

```

```
"comment": {  
  
  "title": "Excelente producto",  
  
  "message": "Llegó antes de lo esperado y en perfectas condiciones. Muy  
recomendado." },  
  
  "metadata": {  
  
    "creation_date": "2018-03-15T14:22:00Z",  
  
    "answer_timestamp": "2018-03-16T09:10:00Z",  
  
    "sentiment": null  
  
  }  
}  
  
// Ejemplo 2 - Reseña solo con score (caso más frecuente - 88% del dataset)  
  
{  
  
  "_id": "990e8400-e29b-41d4-a716-446655440004",  
  
  "review_id": "990e8400-e29b-41d4-a716-446655440004",  
  
  "order_id": "660e8400-e29b-41d4-a716-000000000002",  
  
  "product_id": "660e8400-e29b-41d4-a716-446655440001",  
  
  "score": 4,  
  
  "comment": null,  
  
  "metadata": {  
  
    "creation_date": "2018-05-20T10:45:00Z",  
  
    "answer_timestamp": "2018-05-21T08:30:00Z",  
  
    "sentiment": null  
  
  }  
}
```


Patrones de Modelado

La Tabla 24 documenta los patrones de modelado de MongoDB aplicados en cada colección y su justificación.

Tabla 24

Patrones de modelado MongoDB aplicados en Ecommify

Patrón	Colección	Aplicación	Justificación
Embedded Document	product_catalog	<code>attributes</code> embebido en el documento del producto	Los atributos siempre se consultan junto con el producto, alta cohesión. Evita lookups innecesarios
Embedded Document	product_catalog	<code>images</code> cómo array dentro del documento	Las imágenes se acceden siempre en contexto del producto. Array nativo más eficiente que colección separada
Reference	reviews	<code>order_id</code> y <code>product_id</code> como referencias a PostgreSQL	Las órdenes y productos viven en PostgreSQL. La referencia por UUID compartido permite joins a nivel de aplicación
Subset Pattern	reviews	<code>comment</code> como subdocumento nullable	El 88% de reseñas no tienen comentario. El subset pattern evita documentos con campos vacíos masivos
Extended Reference	product_catalog	<code>product_id</code> igual al UUID de PostgreSQL	Garantiza que los joins a nivel de aplicación entre motores sean deterministas y sin ambigüedad

Justificación PostgreSQL vs MongoDB

La Tabla 25 consolida la decisión arquitectónica final sobre qué entidades residen en cada motor, con criterios técnicos explícitos basados en el Teorema CAP, la naturaleza de los datos y los patrones de acceso identificados.

Tabla 25

Distribución definitiva de entidades entre PostgreSQL y MongoDB

Entidad	Motor actual	Motor futuro	Criterio determinante
geolocations	PostgreSQL	PostgreSQL	Datos estructurados puros. FK desde customers y sellers requiere integridad referencial nativa
customers	PostgreSQL	PostgreSQL	Integridad referencial bidireccional con orders. Inconsistencia = pedido sin cliente
sellers	PostgreSQL	PostgreSQL	FK desde order_items. Joins atómicos con order_payments para facturación
categories	PostgreSQL	PostgreSQL	Tabla pequeña y estable. FK desde products previene productos huérfanos
orders	PostgreSQL	PostgreSQL	Núcleo ACID del sistema. Estados del ciclo de vida requieren transacciones atómicas
order_items	PostgreSQL	PostgreSQL	Entidad débil transaccional. Inserción de orden = orders + order_items + order_payments en una sola transacción
order_payments	PostgreSQL	PostgreSQL	Datos financieros. Consistencia estricta no negociable
products	PostgreSQL	MongoDB (catálogo)	Actualmente en PostgreSQL para simplicidad académica. Atributos heterogéneos justifican migración a MongoDB
product_details	PostgreSQL (JSONB)	MongoDB (embebido)	JSONB es workaround en PostgreSQL. MongoDB modela atributos variables de forma nativa
reviews	PostgreSQL	MongoDB	Texto libre y análisis de sentimiento. Consistencia eventual aceptable. Aggregation Pipeline superior para analytics

Decisiones Arquitectónicas

Las decisiones arquitectónicas de Ecommify no responden a preferencias tecnológicas sino a requisitos técnicos concretos derivados del análisis del dataset Olist y del dominio de comercio electrónico multivendedor. Esta sección documenta la matriz de decisión por entidad, el análisis del Teorema CAP

aplicado, los trade-offs identificados y la estrategia de sincronización entre motores.

Matriz de decisión

La selección del motor de base de datos para cada entidad se evaluó bajo cuatro criterios ponderados: naturaleza de los datos, patrón de acceso predominante, requisitos de consistencia y escalabilidad futura.

La Tabla 26 documenta la decisión arquitectónica por entidad con criterios técnicos explícitos y ponderación por importancia.

Tabla 26

Matriz de decisión arquitectónica PostgreSQL vs MongoDB

Criterio	PostgreSQL	MongoDB	Peso
Integridad referencial (FK)	Nativa, obligatoria entre orders, customers, sellers, order_items	Manual a nivel de aplicación, no garantizada por el motor	Alto
Transacciones ACID	Completas y maduras WAL, MVCC, rollback automático	Parcial: multi-documento desde v4.0 con overhead de coordinación	Alto
Esquema flexible	Rígido, migraciones DDL requeridas, predecible y validado	Documentos anidados sin migrations, ideal para atributos variables	Medio
Escalabilidad horizontal	Limitada nativamente, requiere Citus o Patroni	Sharding nativo, replica sets integrados	Medio
Consultas analíticas	SQL estándar, window functions, CTEs	Aggregation Pipeline	Alto
Atributos variables por categoría	JSONB como workaround	Documentos nativos, anidamiento, arrays	Alto
Costo en plataformas gratuitas	Supabase Free 500MB	Atlas M0 512MB	Medio

Análisis Teorema CAP

El Teorema CAP establece que un sistema de bases de datos distribuido solo puede garantizar simultáneamente dos de tres propiedades: Consistencia (C), Disponibilidad (A) y Tolerancia a Partición (P). Dado que la tolerancia a partición es prácticamente obligatoria en cualquier sistema distribuido real, la elección efectiva se reduce a CP vs AP.

Para Ecommify se evaluó cada módulo según tres criterios: el tipo de datos que maneja, el patrón de acceso predominante y el impacto de una inconsistencia temporal en el negocio. Módulos donde una inconsistencia implica pérdida económica o fraude requieren CP; módulos donde la consistencia eventual no afecta la operación admiten AP.

La Tabla 27 presenta el análisis CAP por módulo de Ecommify.

Tabla 27

Análisis del Teorema CAP por módulo de Ecommify

Módulo	Entidades	Tipo de datos	Patrón de acceso	CAP	Motor
Transaccional	orders, order_items, order_payments	Estructurados	OLTP escrituras frecuentes, lecturas en tiempo real	CP	PostgreSQL
Clientes y vendedores	customers, sellers, geolocations	Estructurados	Lecturas frecuentes, escrituras esporádicas	CP	PostgreSQL
Catálogo de productos	products, product_details, categories	Semi-estructurados	Lecturas masivas, esquema flexible	AP	MongoDB

Reseñas	reviews	Semi-estructurados texto libre	Escrituras altas, lecturas analíticas	AP	MongoDB
Geolocalización	geolocations	Estructurados geoespaciales	Lecturas frecuentes, rara vez escritura	CP	PostgreSQL

Justificación por Módulo

Módulo Transaccional (CP): orders, order_items y order_payments constituyen el flujo de negocio crítico. Un pago registrado sin su orden correspondiente genera problemas legales y financieros. PostgreSQL con transacciones ACID garantiza que ningún estado intermedio quede persistido. La latencia adicional de la consistencia estricta es aceptable porque el usuario espera confirmación de pago.

Módulo Catálogo (AP): products y product_details manejan atributos radicalmente distintos por categoría. Un laptop tiene RAM, procesador y pantalla; unos audífonos tienen impedancia y respuesta de frecuencia. En PostgreSQL esto requiere JSONB como workaround o el anti-patrón EAV. MongoDB lo modela de forma nativa. Si el catálogo tarda 200ms más en actualizarse, el negocio no se ve afectado: la consistencia eventual es aceptable.

Módulo Reviews (AP): las reseñas son texto libre con volumen analítico alto. No participan en transacciones financieras. Una reseña que tarda 2 segundos en aparecer publicada no es un error crítico. MongoDB con su Aggregation Pipeline es superior para análisis de sentimiento, distribución de scores y tendencias por categoría.

Trade-offs y Estrategia de Mitigación

Toda decisión arquitectónica implica trade-offs. La Tabla 28 documenta los trade-offs identificados en la arquitectura híbrida de Ecommify y las estrategias de mitigación aplicadas.

Tabla 28

Trade-offs de la arquitectura híbrida y estrategias de mitigación

Trade-off	Impacto	Estrategia de mitigación
Joins entre motores no son nativos	Para cruzar datos de PostgreSQL y MongoDB se requiere lógica a nivel de aplicación	UUID compartido en product_id garantiza joins deterministas entre motores
Consistencia eventual en MongoDB	Los módulos AP pueden mostrar datos desactualizados transitoriamente	Batch ETL nocturno + event-driven sync reducen la ventana de inconsistencia
Complejidad operacional	Operar dos motores requiere más conocimiento, monitoreo y mantenimiento	Supabase y Atlas M0 abstraen la operación, managed services en entorno académico
Límites de almacenamiento en free tier	Supabase 500MB y Atlas M0 512MB limitan el volumen de datos	Deduplicación de geolocations (~19,000 vs 1,000,163 filas) y carga de subconjunto representativo
Normalización vs rendimiento analítico	Alta normalización (3FN) genera múltiples JOINS en consultas analíticas	Vistas materializadas precalculadas para dashboards OLAP
Migración futura de PostgreSQL a MongoDB	Mover entidades entre motores en producción es costoso	Diseño incremental: JSONB en PostgreSQL como paso intermedio antes de migrar a MongoDB

Estrategia de Sincronización

La sincronización entre el módulo transaccional PostgreSQL y el módulo analítico MongoDB es la pieza clave de la arquitectura híbrida. El principio fundamental es que **PostgreSQL es la fuente de verdad** para todas las entidades

transaccionales, y MongoDB recibe datos derivados o migrados de forma controlada.

La Tabla 29 describe los mecanismos de sincronización propuestos.

Tabla 29

Mecanismos de sincronización entre PostgreSQL y MongoDB

Mecanismo	Descripción	Aplicación	Latencia
Event-driven sync	Trigger en PostgreSQL publica evento al insertar o actualizar products y product_details	Actualización del catálogo en MongoDB product_catalog	Segundos
Batch ETL nocturno	Job programado exporta reviews de PostgreSQL a MongoDB	Migración de reseñas históricas, ejecuta a las 2:00 AM	Horas
UUID compartido	product_id UUID es la misma clave en ambos motores	Joins a nivel de aplicación entre order_items (PostgreSQL) y product_catalog (MongoDB)	N/A
INSERT ON CONFLICT	Scripts de carga idempotentes con ON CONFLICT DO NOTHING	Carga inicial del dataset Olist, permite re-ejecución sin duplicados	N/A

Implementación del trigger de sincronización:

```
-- Función: notifica cambios en products al canal de sincronización

CREATE OR REPLACE FUNCTION notify_product_change()

RETURNS TRIGGER AS $$

BEGIN

    PERFORM pg_notify(

        'product_catalog_sync',

        json_build_object(
```

```

        'operation', TG_OP,

        'product_id', NEW.id,

        'timestamp',

        NOW() )::text );

RETURN NEW;

END;

$$ LANGUAGE plpgsql;

-- Trigger: ejecuta la función después de INSERT o UPDATE en products

CREATE TRIGGER trg_product_catalog_sync

    AFTER INSERT OR UPDATE ON products

    FOR EACH ROW EXECUTE FUNCTION notify_product_change();

```

Implementación del job de sincronización batch:

```

# Job nocturno - Google Colab / servicio programado # Sincroniza reseñas de
PostgreSQL (Supabase) → MongoDB (Atlas)

import pymongo

import psycopg2

from datetime

import datetime def sync_reviews_to_mongodb():

# 1. Conectar a PostgreSQL (Supabase)

    pg_conn = psycopg2.connect(SUPABASE_CONNECTION_STRING)

    pg_cursor = pg_conn.cursor()

# 2. Obtener reseñas creadas en las últimas 24 horas

pg_cursor.execute(""" SELECT id, order_id, score, comment_title, comment_message,
creation_date FROM reviews WHERE creation_date >= NOW() - INTERVAL '1 day' """)

reviews = pg_cursor.fetchall()

```



```
# 3. Conectar a MongoDB Atlas

mongo_client = pymongo.MongoClient(ATLAS_CONNECTION_STRING)

collection = mongo_client['ecommmify']['reviews']

# 4. Upsert por review_id - idempotente ante re-ejecuciones

for review in reviews:

collection.update_one( {'review_id': str(review[0])}, {'$set': { 'review_id':
str(review[0]), 'order_id': str(review[1]), 'score': review[2], 'comment': { 'title':
review[3], 'message': review[4] }, 'metadata': { 'creation_date':
review[5].isoformat(), 'synced_at': datetime.now().isoformat() } }}, upsert=True )

print(f"Sincronizadas {len(reviews):,} reseñas a MongoDB")

# 5. Cerrar conexiones

pg_conn.close() mongo_client.close()
```

Arquitectura Híbrida Final

La Figura 3 y la Figura 4 presentan la arquitectura híbrida final de Ecommify desde dos perspectivas complementarias: la vista lógica de capas y la vista de infraestructura cloud.

Figura 3

Arquitectura híbrida Vista lógica de capas

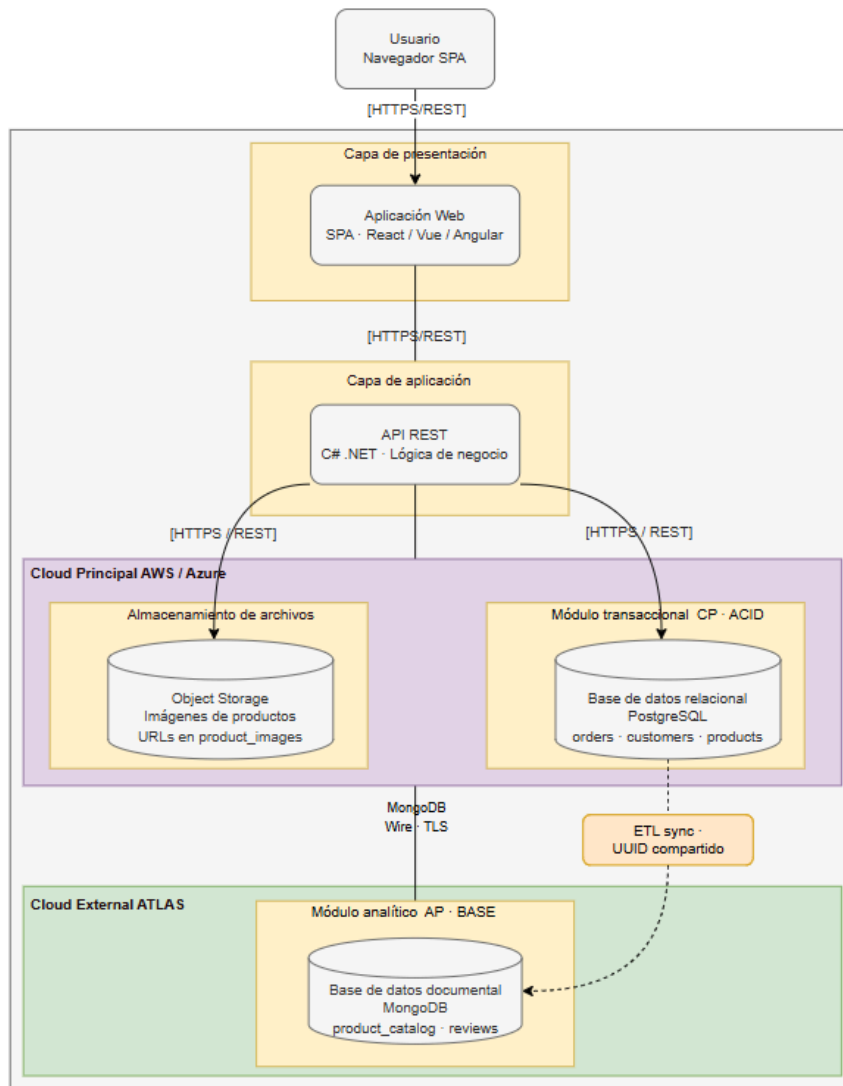
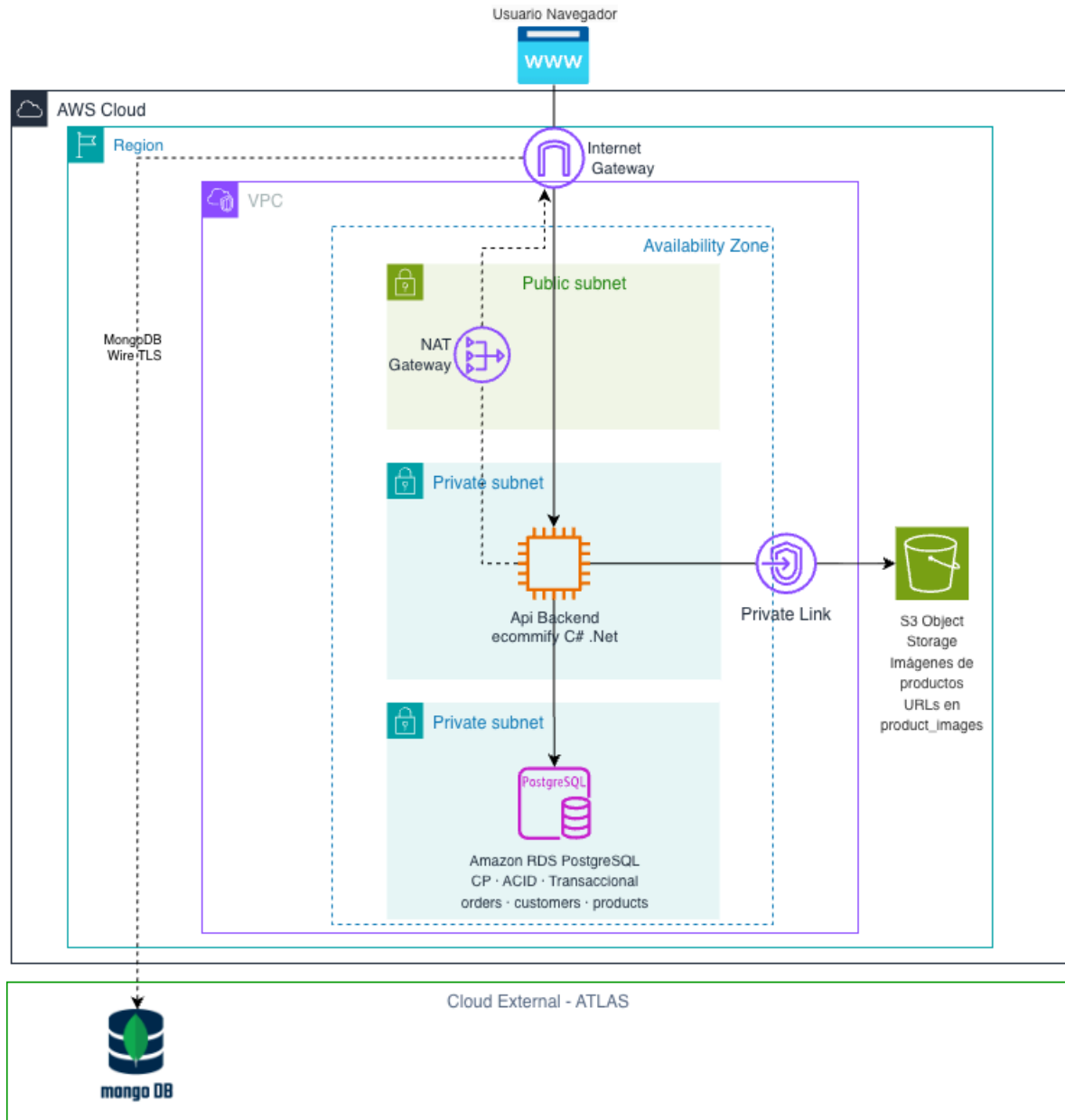


Figura 4

Arquitectura híbrida Vista de infraestructura cloud



La arquitectura híbrida de Ecommify no es una imposición del curso sino la respuesta técnicamente correcta a un sistema que combina requisitos transaccionales estrictos con necesidades analíticas flexibles. PostgreSQL aporta consistencia, integridad y madurez para el corazón del negocio; MongoDB aporta flexibilidad, escalabilidad y potencia analítica para los módulos de alto volumen y

esquema variable. La frontera entre ambos motores está definida por los requisitos de negocio, no por preferencias tecnológicas.

Estrategia OLTP / OLAP

Ecommify opera bajo dos patrones de carga de datos fundamentalmente distintos que coexisten en la misma plataforma: cargas transaccionales de alta frecuencia que requieren consistencia inmediata, y cargas analíticas intensivas que priorizan rendimiento de lectura sobre grandes volúmenes de datos históricos. El diseño de la base de datos considera ambos patrones de forma explícita para garantizar que ninguno degrade al otro.

Cargas transaccionales OLTP

El módulo OLTP de Ecommify gestiona las operaciones del flujo comercial en tiempo real. Estas operaciones se caracterizan por escrituras frecuentes, transacciones cortas y requisitos estrictos de consistencia.

La Tabla 30 documenta las operaciones OLTP principales de Ecommify con sus patrones de acceso y optimizaciones aplicadas.

Tabla 30

Operaciones OLTP principales de Ecommify

Operación	Tablas involucradas	Frecuencia	Optimización aplicada
Crear orden de compra	orders + order_items + order_payments	Alta	Transacción ACID atómica, las tres inserciones en un solo bloque BEGIN/COMMIT
Actualizar estado de orden	orders	Alta	Índice en order_status + update_at con trigger

Registrar pago	order_payments	Alta	FK order_id con índice, validación CHECK payment_value >= 0
Consultar historial de cliente	orders + order_items	Media	Índice en customer_id, partition pruning en orders_2026
Registrar reseña	reviews	Media	Índice en order_id
Actualizar catálogo	products + product_details	Baja	GIN index en attributes para búsquedas JSONB

Transacción OLTP representativa (Crear orden):

```

BEGIN;

-- 1. Insertar la orden

INSERT INTO orders (

    id, customer_id, order_status, purchase_timestamp, estimated_delivery_date,

    update_at

) VALUES ( gen_random_uuid(), $1, 'created', NOW(), NOW() + INTERVAL '7 days',

    NOW()

)

RETURNING id INTO v_order_id;

-- 2. Insertar los ítems

INSERT INTO order_items ( order_id, item_sequence, product_id, seller_id, price,

freight_value

)

SELECT v_order_id, ROW_NUMBER() OVER(), product_id, seller_id, price, freight_value

FROM unnest($2::order_item_input[]);

-- 3. Registrar el pago

```

```
INSERT INTO order_payments ( id, order_id, payment_type, payment_installments,
payment_value, update_at

) VALUES ( gen_random_uuid(), v_order_id, $3, $4, $5, NOW() );

COMMIT;

-- Si cualquier paso falla → ROLLBACK automático
```

Cargas analíticas - OLAP

El módulo OLAP de Ecommify soporta consultas de negocio sobre grandes volúmenes de datos históricos para dashboards, reportes de ventas y análisis de comportamiento de clientes. Estas consultas se caracterizan por lecturas sobre múltiples tablas, agregaciones complejas y tolerancia a datos con cierto retraso.

La Tabla 31 documenta las consultas analíticas principales y su estrategia de optimización.

Tabla 31

Consultas analíticas principales de Ecommify

Consulta	Tablas involucradas	Estrategia de optimización
Ventas por categoría por mes	orders + order_items + products + categories	Vista materializada mv_sales_by_category_monthly
Segmentación de clientes por volumen	orders + order_payments + customers	Vista materializada mv_customer_segments
Desempeño de vendedores	order_items + sellers + orders	Vista materializada mv_seller_performance
Distribución geográfica de ventas	orders + customers + geolocations	Índice en geolocation_id + partition pruning
Análisis de scores de reseñas	reviews	MongoDB Aggregation Pipeline (fase futura)

Materialized Views

Las vistas materializadas son la pieza central de la estrategia OLAP de Ecommify.

Estas pre-calculan y almacenan en disco el resultado de consultas analíticas complejas, permitiendo que los dashboards respondan en milisegundos en lugar de segundos al evitar recalcular sobre 100,000 órdenes en cada consulta.

Vista materializada 1 → Ventas por categoría por mes:

```
CREATE MATERIALIZED VIEW mv_sales_by_category_monthly AS

SELECT

    c.name AS category_name,

    DATE_TRUNC('month', o.purchase_timestamp) AS month,

    COUNT(DISTINCT o.id) AS total_orders,

    COUNT(oi.order_id) AS total_items,

    SUM(oi.price) AS total_revenue,

    AVG(oi.price) AS avg_price,

    SUM(oi.freight_value) AS total_freight

FROM orders o

JOIN order_items oi ON o.id = oi.order_id

JOIN products p ON oi.product_id = p.id

JOIN categories c ON p.category_id = c.id

WHERE o.order_status = 'delivered'

GROUP BY c.name, DATE_TRUNC('month', o.purchase_timestamp)

ORDER BY month DESC, total_revenue DESC WITH DATA;

-- índice para consultas por categoría y mes
```

```
CREATE INDEX idx_mv_sales_category ON mv_sales_by_category_monthly(category_name,
month);
```

Vista materializada 2 → Segmentación de clientes:

```
CREATE MATERIALIZED VIEW mv_customer_segments AS

SELECT

    cu.id AS customer_id,

    cu.name AS customer_name,

    g.city, g.state,

    COUNT(DISTINCT o.id) AS total_orders,

    SUM(op.payment_value) AS total_spent,

    AVG(op.payment_value) AS avg_order_value,

    MAX(o.purchase_timestamp) AS last_purchase,

    CASE

        WHEN SUM(op.payment_value) > 1000 THEN 'Premium'

        WHEN SUM(op.payment_value) > 500 THEN 'Regular'

        ELSE 'Ocasional' END AS segment

FROM customers cu

JOIN orders o ON cu.id = o.customer_id

JOIN order_payments op ON o.id = op.order_id

LEFT JOIN geolocations g ON cu.geolocation_id = g.id

GROUP BY cu.id, cu.name, g.city, g.state WITH DATA;

-- Índice para filtrar por segmento y estado
```



```
CREATE INDEX idx_mv_customer_segment ON mv_customer_segments(segment, state);
```

Vista materializada 3 → Desempeño de vendedores:

```
CREATE MATERIALIZED VIEW mv_seller_performance AS

SELECT

    s.id AS seller_id,

    s.name AS seller_name,

    g.city AS seller_city,

    g.state AS seller_state,

    COUNT(DISTINCT oi.order_id) AS total_orders,

    COUNT(oi.order_id) AS total_items_sold,

    SUM(oi.price) AS total_revenue,

    AVG(oi.price) AS avg_item_price,

    AVG(oi.freight_value) AS avg_freight,

    ROUND(AVG(r.score), 2) AS avg_review_score,

    COUNT(r.id) AS total_reviews

FROM sellers s

JOIN order_items oi ON s.id = oi.seller_id

JOIN orders o ON oi.order_id = o.id

LEFT JOIN reviews r ON o.id = r.order_id

LEFT JOIN geolocations g ON s.geolocation_id = g.id

WHERE o.order_status = 'delivered' GROUP BY s.id, s.name, g.city, g.state

WITH DATA;
```

```
CREATE INDEX idx_mv_seller_revenue

ON mv_seller_performance(total_revenue DESC);

CREATE INDEX idx_mv_seller_state

ON mv_seller_performance(seller_state);
```

Plan de Mantenimiento

El mantenimiento de la base de datos garantiza que las vistas materializadas reflejen datos actualizados, que el almacenamiento no crezca indefinidamente y que el rendimiento no degrade con el tiempo. La Tabla 32 documenta el plan de mantenimiento programado para Ecommify.

Tabla 32

Plan de mantenimiento programado de Ecommify

Job	Frecuencia	Horario	Comando	Justificación
VACUUM de orders	Diario	2:00 AM	VACUUM ANALYZE orders	Libera espacio de filas eliminadas y actualiza estadísticas del planner
REFRESH mv_sales_by_category_monthly	Diario	2:30 AM	REFRESH MATERIALIZED VIEW mv_sales_by_category_monthly	Dashboard de ventas se actualiza cada día con datos del día anterior
REFRESH mv_seller_performance	Diario	2:45 AM	REFRESH MATERIALIZED VIEW mv_seller_performance	Reportes de vendedores actualizados diariamente
REFRESH mv_customer_segments	Semanal	Domingos 3:00 AM	REFRESH MATERIALIZED VIEW mv_customer_segments	La segmentación de clientes no cambia radicalmente en 24 horas
VACUUM FULL tablas	Mensual	Primer domingo	VACUUM FULL ANALYZE order_items	Compactación completa para recuperar espacio fragmentado

grandes		4:00 AM		
Creación de nueva partición	Mensual	Último día del mes	CREATE TABLE orders_YYYY PARTITION OF orders FOR VALUES FROM (...)	Preparar la partición del mes siguiente antes de que lleguen los datos
Sync reviews, MongoDB	Diario	Diario	Script Python batch ETL	Mantener MongoDB actualizado con reseñas del día anterior

Implementación con pg_cron en Supabase:

```
-- Activar extensión pg_cron

CREATE EXTENSION IF NOT EXISTS pg_cron;

-- VACUUM diario a las 2:00 AM

SELECT cron.schedule(

    'vacuum-orders-daily',

    '0 2 * * *',

    'VACUUM ANALYZE orders'

);

-- Refresh mv_sales diario a las 2:30 AM

SELECT cron.schedule(

    'refresh-mv-sales',

    '30 2 * * *',

    'REFRESH MATERIALIZED VIEW mv_sales_by_category_monthly'

);

-- Refresh mv_seller diario a las 2:45 AM

SELECT cron.schedule(

    'refresh-mv-seller',
```

```

'45 2 * * *',

'REFRESH MATERIALIZED VIEW mv_seller_performance'

);

-- Refresh mv_customers semanal domingos 3:00 AM

SELECT cron.schedule(

    'refresh-mv-customers',

    '0 3 * * 0',

    'REFRESH MATERIALIZED VIEW mv_customer_segments'

);

```

Escalabilidad

La arquitectura de Ecommify fue diseñada para crecer progresivamente sin cambios disruptivos en el núcleo del sistema. La Tabla 33 documenta la estrategia de escalabilidad por capa y el umbral que activaría cada acción.

Tabla 33

Estrategia de escalabilidad por capa de Ecommify

Capa	Límite actual	Umbral de acción	Estrategia de escalamiento
PostgreSQL Supabase Free	500 MB	400 MB ocupados	Migrar a Supabase Pro (8GB) o AWS RDS con almacenamiento elástico
MongoDB Atlas	512 MB (M0)	400 MB ocupados	Migrar a Atlas M10 con almacenamiento configurable y sharding disponible
Particionamiento orders	Partición anual	> 1M órdenes por año	Cambiar a particiones mensuales para partition pruning más granular
Vistas materializadas	Refresh diario	Refresh tarda > 5 min	Implementar REFRESH CONCURRENTLY para no

			bloquear lecturas durante el refresh
Catálogo de productos	PostgreSQL JSONB	> 100K productos con atributos	Migrar product_details a colección product_catalog en MongoDB
Reseñas	PostgreSQL	> 500K reseñas	Migrar reviews a colección MongoDB con Aggregation Pipeline para analytics
API Backend	Instancia única	Latencia p95 > 200ms	Escalar horizontalmente con load balancer y múltiples instancias del API

Métricas de monitoreo OLTP y OLAP:

```
-- Monitoreo OLTP: tiempo promedio de transacciones
```

```
SELECT
```

```
    query,
```

```
    calls,
```

```
    ROUND(mean_exec_time::numeric, 2) AS avg_ms,
```

```
    ROUND(total_exec_time::numeric, 2) AS total_ms
```

```
FROM pg_stat_statements
```

```
WHERE query ILIKE '%orders%'
```

```
ORDER BY mean_exec_time DESC
```

```
LIMIT 10;
```

```
-- Monitoreo OLAP: tamaño de vistas materializadas
```

```
SELECT
```

```
    schemaname,
```

```
    matviewname,
```

```
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||matviewname)) AS size
```

```
FROM pg_matviews
```

```

ORDER BY pg_total_relation_size(schemaname||'.'||matviewname) DESC;

-- Monitoreo general: uso de almacenamiento por tabla

SELECT

    tablename,

    pg_size_pretty(pg_total_relation_size(tablename::regclass)) AS size

FROM pg_tables

WHERE schemaname = 'public'

ORDER BY pg_total_relation_size(tablename::regclass) DESC;

```

Repositorio GitHub

<https://github.com/lara44/sabana.ecommify.database.design.git>

Conclusiones

El desarrollo de la actividad del caso de uso Ecommify deja una reflexión profunda: no hay un único motor de base de datos que permita controlar todos los dominios que presenta un e-commerce. La decisión arquitectónica de seguir un enfoque híbrido se debe a los patrones de acceso y a las garantías necesarias para cada módulo de negocio. Se seleccionó en primer lugar el motor de base de datos PostgreSQL debido a que el dominio del negocio requiere una integridad referencial estricta con operaciones atómicas de alta complejidad. El modelo de negocio propuesto no puede permitir tolerar inconsistencias en pedidos, pagos y clientes porque cualquier fallo de integridad de datos en este dominio puede tener un impacto directo en la operación y en la confianza de los clientes.

El análisis exploratorio del dataset Olist fue el punto de partida que fundamentó todas las decisiones de diseño posteriores. Antes de definir cualquier tabla o relación, los datos revelaron patrones que no eran evidentes en la documentación: 2.965 órdenes sin fecha de entrega confirmada, 610 productos sin categoría asignada y más de un millón de registros de geolocalización con múltiples coordenadas por código postal. Cada uno de estos hallazgos se tradujo directamente en una decisión de diseño concreta, campos nullable justificados, claves foráneas opcionales y la adopción de UUID sintético en geolocations en lugar de una clave natural ambigua. Este proceso confirmó que la normalización no es un ejercicio teórico sino una respuesta a problemas reales de calidad de datos: aplicar 1FN, 2FN y 3FN sobre la tabla denormalizada OrdersFull nos permitió identificar exactamente dónde vivían las redundancias y por qué generaban inconsistencias. El proyecto demostró que un buen diseño de base de datos empieza siempre por entender los datos antes de modelarlos.

Algo que nos mostró este ejercicio es que las decisiones de arquitectura de datos no deben basarse únicamente en preferencias tecnológicas, sino en el comportamiento real de los datos, los patrones de acceso y las restricciones operacionales del dominio. El análisis exploratorio permitió transformar observaciones del dataset en decisiones concretas de modelado, evidenciando que una arquitectura híbrida correctamente delimitada puede equilibrar consistencia transaccional y flexibilidad analítica sin comprometer la evolución futura del sistema.

Referencias

Bradshaw, S., Brazil, E., & Chodorow, K. (2019). *MongoDB: The definitive guide* (3.^a ed.). O'Reilly Media.

Brewer, E. A. (2000). *Towards robust distributed systems*. Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC).
<https://doi.org/10.1145/343477.343502>

MongoDB, Inc. (2025). *Data modeling introduction*. MongoDB Manual.
<https://www.mongodb.com/docs/manual/core/data-modeling-introduction/>

MongoDB, Inc. (2025). *MongoDB Atlas documentation*.
<https://www.mongodb.com/docs/atlas/>

Olist. (2018). *Brazilian E-Commerce Public Dataset by Olist* [Conjunto de dados]. Kaggle. <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

PostgreSQL Global Development Group. (2025a). *Database design*. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/ddl.html>

PostgreSQL Global Development Group. (2025b). *Data types*. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/datatype.html>

PostgreSQL Global Development Group. (2025c). *Server programming: Extending SQL*. PostgreSQL 16 Documentation.
<https://www.postgresql.org/docs/16/extend.html>

PostgreSQL Global Development Group. (2025d). *Table partitioning*.

PostgreSQL 16 Documentation.

<https://www.postgresql.org/docs/16/ddl-partitioning.html>

PostgreSQL Global Development Group. (2025e). *Materialized views*.

PostgreSQL 16 Documentation.

<https://www.postgresql.org/docs/16/rules-materializedviews.html>

Supabase, Inc. (2025). *Supabase documentation*.

<https://supabase.com/docs>